

Faster Synthesis of Combinational Logic Based on Multiplexer Trees and Binary Decision Diagrams

Lukas KOHUTKA, Peter PISTEK*
 * Slovak University of Technology in Bratislava
 Faculty of Informatics and Information Technologies
 Ilkovicova 2, 842 16 Bratislava, Slovakia
 kohutka555@gmail.com

Abstract—Multiplexers are well known as a basic building element of digital and mixed signal circuits thanks to their ability to perform any Boolean function. Optimization is a significant part of synthesis of combinational logic, since performance has to be improved, area and power consumption have to be reduced. The paper presents a novel faster optimization method for multiplexer trees using basic BDD reduction methods, residual variables, a hash table and top-down approach. An option to automatically replace some multiplexers in the multiplexer tree with basic logic gates has been added in order to achieve better results. This method also works with multiple Boolean functions at once so that we can design circuits with more than one output. Experimental results show that implemented algorithm reduces total amount of multiplexers in optimized multiplexer tree by up to 99,99% in comparison to non-optimized multiplexer tree. In addition up to 63,46% of multiplexers can be replaced with a logic gate OR, AND or XOR, which can reduce total amount of transistors needed to realise given combinational logic by up to 24,23%.

Keywords—combinational logic synthesis; binary decision diagram; multiplexer tree; hash table; logic gate

I. INTRODUCTION

Multiplexers are common building blocks for datapaths and they typically account for over 25% of chip size [12]. Functional blocks, as blocks which contain also multiplexers, can be easily disconnected from power supply based on designed power architecture integrated in a system-level model [8]. Multiplexers are also used in combinational circuit redesign for increasing quality of time specification testing and therefore they have to be involved there in optimal overhead or another logic gates have to be used [9]. Multiplexers are also used in self-testing or self-repairing memories [10] where it is important to minimize their usability in controlling test and functional modes and also power consumption. However, it is necessary to design these blocks as effective as possible. In multiplexer circuits it is possible usage of dynamic propagation path control [11] to decrease its dynamic power consumption. To reduce size on a chip and static power consumption of multiplexer tree, the similarity between multiplexer tree structures (built from 2-to-1 multiplexers) and Binary Decision Diagrams [2-3] (further BDD) is used. Therefore, BDD optimization methods can be used to optimize multiplexer tree in combination with other optimization methods. This method combines basic BDD optimization methods [4]

with replacing some multiplexers by input signals (known as residual variables) and optionally by basic logic gates. Hash table and top-down approach speed up this method. Our research has been focused on minimization of amount of logic components required to create desired circuit and minimization of time required to design this circuit.

The organization of this paper is following. Section II contains theory of BDDs and basic BDD optimization methods. In Section III methods improving performance of proposed method are described. Section IV describes rules for replacing multiplexers by basic logic gates or even residual variables. Section V contains a description of simulated annealing based variable ordering algorithm. Section VI shows experimental results achieved by the proposed improvement methods. The last section concludes the paper and summarises benefits of this method.

II. BASIC BDD OPTIMIZATION

BDD optimization methods can be divided into two main categories [1]:

- 1) BDD ordering – results in Ordered Binary Decision Diagram (OBDD), which respects a given order of input variables. Variable ordering can have great impact on effectiveness of BDD reduction.
- 2) BDD reduction – when applied on OBDD, results in Reduced OBDD (ROBDD) which has lower number of nodes than not reduced BDD. ROBDD respects these two rules:
 - a) Uniqueness – no two distinct nodes u and v represent the same variable and have the same left and right successor, i.e.:
 $var(u) = var(v)$, $left(u) = left(v)$, $right(u) = right(v)$ what implies $u = v$
 - b) Non-redundancy – no variable node u has identical left and right successor, i.e.:
 $left(u) = right(u)$

The proposed method uses BDD reduction described above. BDD ordering is a complex problem because the total number of all possible orderings of input variables is equal to factorial of input variables number. This problem was not aim of this paper therefore the proposed method uses implicit variable ordering defined by input Boolean function in a form of vector. Up to all possible variable orderings can be tried by changing the input vector and creating ROBDD from each vector. The proposed method

uses a parameter c , which defines a depth of searching the best variable ordering. Value of c can be in range from 0 to n , where n is the number of input variables. The bigger the c is, the more complex the whole synthesis process is. For example, if we choose c to be 0, the BDD ordering problem is solved in constant time (only one implicit variable order is used), if we choose c to be 1, the BDD ordering problem is solved in linear time (n variable orderings are used), c equal to 2 stands for quadratic complexity, c equal to 3 stands for cubic complexity, up to c equal to n , which means use of all possible variable orderings (factorial complexity).

Although the variable ordering was not the aim of this paper, we also designed a simulated annealing based algorithm, which represents the best compromise between the quality of result and time complexity of the algorithm. This algorithm is described in section V.

III. TOP-DOWN APPROACH AND HASH TABLES

Performance of whole synthesis process can be improved by these two techniques: top-down approach for building the BDD and hash tables for uniqueness of nodes in the BDD.

The top-down approach means that the creation of BDD is performed in order from top (beginning from root) down to successors. During this creation, both reduction rules “uniqueness” and “non-redundancy” are checked at each BDD node even before the creation of the node. When one of those reduction rules should apply and thus reduce the BDD by removing the node, this node is not even created. This means that we don’t need to pass by all paths of BDD because some of the paths are closed sooner. We also do not need to allocate so much memory as we would need in the bottom-up approach. The top-down approach results in fewer amount of nodes needed to check with reduction rules, which results in faster creation of OBDD and thus faster synthesis of combinational logic based on the OBDD.

The second improvement technique is based on a hash table, which is used in order to greatly improve performance of reduction rule named “uniqueness”. After the successful creation of each node a hash index from this node by a hash function is generated. Our hash function generates the hash index by using a XOR on every k bits of a node’s vector so the resulting hash index is a k -bit number and the hash table has size of 2^k pointers, where each pointer represents one linked list of pointers on BDD nodes. Of course, other hash function implementations may be used. A choice of a hash function was not the aim of this paper. Every created node (which means a pointer pointing to the node) is added to the linked list situated in the hash table at index computed by the hash function. The performance improvement is achieved by the fact that with help of the hash table we do not need to compare the new node with all existing nodes of the same depth (same control variable) in the ROBDD, but we only need to compare the new node with the nodes that are in the hash table at the same hash index. More than one hash table can be used and thus even less nodes could be needed to compare. Potentially, we could have one hash table for

each control variable and thus the maximum number of hash tables is the number of input variables. The nodes with the lowest 4 control variables can use simple arrays instead of the hash tables because the size of node’s vector is small enough.

IV. REPLACEMENT OF MULTIPLEXERS

The smallest ROBDD is transformed to multiplexer tree and can be improved by replacing some of the multiplexers with direct input variable or negated input variable (in both cases these signals are called “residual variable”) and some other multiplexers can be replaced with basic logic gate.

In fact, the “residual variable” replacement can be used always and already during the creation of ROBDD.

Let u to be a MUX, $left(u)$ to be a left data input of u , let $right(u)$ to be a right data input of u , let $r(u)$ to be a control input of u and $l(u)$ to be an inverted control input of u . Let ‘0’ to be a constant zero value and ‘1’ to be a constant one value, then rules for MUX replacement are as follows:

- If $left(u) = '0'$ and $right(u) = '1'$, then u can be replaced by $r(u)$. (residual variable)
- If $left(u) = '1'$ and $right(u) = '0'$, then u can be replaced by $l(u)$. (residual variable)
- If $left(u) = '0'$ and $right(u)$ is neither ‘0’ nor ‘1’, then u can be replaced by logic gate $AND(right(u), r(u))$.
- If $right(u) = '0'$ and $left(u)$ is neither ‘0’ nor ‘1’, then u can be replaced by logic gate $AND(left(u), l(u))$.
- If $left(u) = '1'$ and $right(u)$ is neither ‘0’ nor ‘1’, then u can be replaced by logic gate $OR(right(u), l(u))$.
- If $right(u) = '1'$ and $left(u)$ is neither ‘0’ nor ‘1’, then u can be replaced by logic gate $OR(left(u), r(u))$.
- If $left(u) =$ input variable v and $right(u) =$ negated input variable $\neg v$, then u can be replaced by logic gate $XOR(v, r(u))$.
- If $right(u) =$ input variable v and $left(u) =$ negated input variable $\neg v$, then u can be replaced by logic gate $XOR(v, l(u))$.

V. SIMULATED ANNEALING BASED VARIABLE ORDERING

We also looked at the variable ordering problem because the variable ordering can have massive impact on the effectiveness of the reduction of BDD.

The deterministic factorial complexity is simply too high for practical use. The factorial variable ordering is too time-consuming for Boolean functions with more than 10 input variables.

The constant and linear variable ordering algorithms are fast enough but may not result in effective combinational logic design in some circumstances.

This was the reason to provide the best compromise between these two approaches, which is achieved by the simulated annealing algorithm. This algorithm belongs to

stochastic optimization algorithms and is very similar to the genetic algorithms and the hillclimbing algorithm. All of these algorithms contain a mutation operation, which represents a change in variable ordering in our case. The proposed simulated annealing algorithm uses a mutation, which changes always only those variables that are neighbours (i.e. are directly next to each other). For example, if we have a variable ordering 1 2 3 4 5, we can mutate it to 2 1 3 4 5, but we cannot mutate it to 3 1 2 4 5.

This means that every variable ordering can mutate exactly in $n - 1$ ways, where n is the number of variables. The proposed algorithm mutates with some probability to the best possible solution from the $n - 1$ possible ways. A worse mutation can be used with the inverse probability. The probability changes in time so that in the end we select only the best possible mutations (which stands for the hillclimbing algorithm).

The simulated annealing algorithm uses some parameters typical to this algorithm, which should be optimised too. These parameters are:

- initial temperature
- decrease of temperature
- duration of temperature

Our algorithm used the initial temperature equal to 30, decrease of temperature equal to 5 and the duration of temperature equal to 100. Change of these parameters may greatly affect the performance and quality of the algorithm. One of the possible ways to optimise these parameters is an approach according to the experimental results.

VI. EXPERIMENTAL RESULTS

During testing, we used IWLS'93 Benchmark Set: Version 4.0. The testing was mainly aimed for evaluating the time needed for synthesis and rate of reduction (number of nodes after reduction in comparison to number of nodes before reduction).

We tested several possibilities for variable ordering complexity c (described in Section II) – 0 (constant), 1 (linear) and n (factorial). The hash function uses $k = 16$.

There are three tables (Table I-III), each for one type of variable ordering complexity. First column displays the name of benchmark circuit. Column “Before optimization” shows number of nodes in BDD before optimization. Next column “After optimization” shows number of nodes in optimized BDD with BDD reduction (Section II) and all residual variables (Section IV). Column named “Reduction ratio” shows percent improvement of optimized circuit in comparison to original circuit. Column “Logic gates” shows number of nodes after optimization, which can be implemented with use of basic logic gates. Next column “Logic gate ratio” shows the efficiency of our MUX-1 replacement by logic gates in optimized multiplexer tree. The last column “Time (ms)” displays total real time in milliseconds required by this synthesis process.

The last table with factorial complexity does not contain all benchmark circuits because some of them had

too much input variables for factorial complexity and thus the time required by synthesis process was too long.

Table I shows that even constant variable ordering offers very good reduction ratio, many times more than 74,19%. This method is suitable as a basic element for other algorithms, which combine the ROBDD with other more sophisticated methods for variable ordering problem.

The results of linear complexity (Table II) show quite good improvement in some cases and the time needed for synthesis process is still relative short. For example amount of nodes after optimization for circuit “cordic” is 62,53% smaller than the result of constant complexity. This algorithm can be used as a basic element in other more sophisticated methods too, for example using dynamic approaches on each ROBDD created by this linear algorithm.

Bigger complexities (Table III) are suitable for only small number of input variables, for example factorial variable ordering is worth to be used for Boolean functions with maximum of 10 input variables. The more input variables the function has, the less complex variable ordering should be used. Even constant variable ordering reaches relative high reduction ratios when used for bigger Boolean functions (cordic - 99,94%), so that more complex variable orderings do not often bring big improvements and thus are not worth using them.

The experimental results also show that speed-up improvements based on hash table and top-down approach allow us to synthesize circuits with more input variables or to use more complicated algorithm for solving variable ordering problem in order to achieve even more optimized combinational circuit.

The testing has shown that even after optimization can be still relative big proportion of nodes used as basic logic gates instead of multiplexers. Sometimes the amount of logic gates is even higher than the number of multiplexers. According to the experimental results, the logic gate ratio generally decreases as we increase the number of input variables. On the other hand, this rate depends markedly on the Boolean function itself. For example circuit “cordic” uses 23 input variables and the logic gate ratio is 31,06% (Table I). We need generally 10 transistors for each MUX-1 in CMOS technology. If we replace multiplexer with OR-2 or AND-2 gate, we need only 6 transistors (amount of transistors is reduced by 40%). In case of replacing a MUX-1 with XOR-2 gate the number of transistors needed falls from 10 to 8, which means another 20% reduction rate. For example amount of transistors needed to realise circuit “rd53” can be reduced by 15%.

The last table shows a comparison of effectiveness between the proposed simulated annealing algorithm and other algorithms like the constant, linear, factorial and the hillclimbing. As we can see, the simulated annealing algorithm has results relative very close to the results of the factorial algorithm. However, the simulated annealing algorithm has much lower computation complexity, which is only quadratic.

VII. CONCLUSION

In this paper, a new method for automatic synthesis and optimization of combinational logic based on BDD is presented. Such BDD represents a highly optimized multiplexer tree, what is achieved by using basic BDD optimization methods and after that, the method of residual variables is applied. A multiplexer tree obtained through the conversion of a BDD generated by this algorithm may require additional inverter circuit to provide residual variable in both positive and negative forms. The area reduced by our algorithm is significantly greater compared to this addition, and since multiplexers are far more complex, this inverter circuit can be neglected.

The main benefits of this new method are a better speed of the synthesis process that can be seen mainly at Boolean functions with higher number of input variables. This is caused by using the hash tables and the top-down approach, which removes the BDD nodes as soon as possible and avoids passing by all paths of BDD. The second benefit is the possibility to replace some multiplexers with the basic logic gates AND, OR and XOR. For each replaced multiplexer exactly one of these logic gates is always used. For example 41,67% of multiplexers can be replaced in "rd53", which results in 15% less transistors in CMOS technology.

The computational requirements of this algorithm are relative low thanks to the hash table and top-down approach. Our method can be used in other algorithms aimed for solving variable ordering problem like dynamical ordering optimization algorithms based on heuristics [4-7] or the simulated annealing algorithm, which is proposed in this paper too. The simulated annealing proved to be the best compromise because it gives us results very close to the factorial algorithm but for the much cheaper price in terms of the computation time. The simulated annealing has only quadratic time complexity.

The output of the implemented program (which served for verification of the proposed algorithms) is in a form of VHDL or Verilog and thus can be further synthesized in various technologies, where some can prefer only multiplexers and some can prefer the replacement of multiplexers by basic logic gates.

This method is also able to use multiple Boolean functions in order to design combinational logic with more than one output bit.

In future work, we plan to add another optimization method based on merging equivalent subtrees from

multiple binary decision diagrams with help of using one common hash table.

ACKNOWLEDGEMENT

This work was partially supported by the Slovak Science Grant Agency (VEGA 1/0616/14 "Optimization of low-power design of digital and mixed integrated systems").

REFERENCES

- [1] R. Ebendt, G. Fey, R. Drechsler, "Advanced BDD Optimization," Springer, 2005, 222p, ISBN 978-0-387-25453-1.
- [2] K. S. Brace, R. E. Bryant, R. L. Rudell, "Efficient Implementation of a BDD Package," In: 27th ACM/IEEE Design Automation Conference, IEEE, 1990, pp. 40-45.
- [3] P. Pišteš, M. Kolesár, K. Jelemenská, "Optimization of multiplexer trees using modified truth table," In: 2010 International Conference on Applied Electronics (AE), Pilsen: IEEE, 2010, pp. 265-268.
- [4] P. W. C. Prasad, et al., "Binary Decision Diagrams: An Improved Variable Ordering Using Graph Representation of Boolean Functions," International Journal of Computational Science, 1(1): pp.1-7, 2006.
- [5] R. Drechsler, "Evaluation of Static Variable Ordering Heuristics for MDD Construction," ISMVL 2002: pp.254-260, 2002.
- [6] I. Furdu, B. Patrut, "Genetic Algorithm for Ordered Decision Diagrams Optimization," Proceedings of ICMI 45, 2006, pp. 437-444.
- [7] I. Furdu, B. Patrut, "Genetic Algorithm for Ordered Decision Diagrams Optimization," Proceedings of ICMI 45, 2006, pp. 437-444.
- [8] D. Macko, K. Jelemenská, "Managing digital-system power at the system level," in Proc. of IEEE Africon 2013 Sustainable Engineering for a Better Future, 2013.
- [9] M. Siebert, E. Gramatova, "Delay Fault Coverage Increasing in Digital Circuits," in Proc. of Euromicro Conference on Digital System Design (DSD), 2013, pp. 475-478.
- [10] S. Kristofik, E. Gramatova, "Redundancy algorithm for embedded memories with block-based architecture," in Proc. of IEEE 16th International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS), 2013, pp. 271-274.
- [11] N. S. Li, J. D. Huang, and H. J. Huang, "Low Power Multiplexer Tree Design Using Dynamic Propagation Path Control," in Proc. of IEEE APCCAS, Dec. 2008, pp. 838-841.
- [12] P. Metzgen and D. Nancekievill, "Multiplexer restructuring for FPGA implementation cost reduction," in Proc. of the 42nd Annual Design Automation Conference, ser. DAC '05, 2005, pp. 421-426.

TABLE I.
EXPERIMENTAL RESULTS(1)

Benchmark Circuit	Constant complexity of variable ordering problem					
	Before optimization	After optimization	Reduction ratio	Logic gates	Logic gate ratio	Time (ms)
con1	94	15	84,04%	7	46,67%	2
rd53	93	24	74,19%	10	41,67%	4
rd84	1020	70	93,14%	22	31,43%	5
sqrt8	336	39	88,39%	15	38,46%	5
9sym	511	31	93,93%	10	32,26%	2
sao2	4092	188	95,41%	88	46,81%	11
cm152a	2047	9	99,56%	2	22,22%	1
cm151a	8190	1018	87,57%	78	7,66%	37
cm150a	1966082	131069	93,75%	549	0,42%	2671
parity	65535	3419	94,78%	2	0,06%	90
t481	65535	607	99,07%	174	28,67%	49
5xp1	628	89	85,83%	29	32,58%	13
misex1	473	52	89,00%	33	63,46%	9
inc	1015	90	91,13%	54	60,00%	12
squar5	248	38	84,68%	23	60,53%	8
xor5	31	7	77,42%	2	28,57%	1
cordic	16777214	10403	99,94%	3231	31,06%	3295

TABLE II.
EXPERIMENTAL RESULTS(2)

Benchmark Circuit	Linear complexity of variable ordering problem					
	Before optimization	After optimization	Reduction ratio	Logic gates	Logic gate ratio	Time (ms)
con1	94	13	86,17%	7	53,85%	9
rd53	93	24	74,19%	10	41,67%	16
rd84	1020	70	93,14%	22	31,43%	39
sqrt8	336	37	88,99%	14	37,84%	24
9sym	511	31	93,93%	10	32,26%	13
sao2	4092	154	96,24%	83	53,90%	69
cm152a	2047	9	99,56%	2	22,22%	9
cm151a	8190	30	99,63%	16	53,33%	54
cm150a	1966082	31	99,99%	16	51,61%	16572
parity	65535	3419	94,78%	2	0,06%	494
t481	65535	314	99,52%	87	27,71%	274
5xp1	628	76	87,90%	21	27,63%	48
misex1	473	48	89,85%	27	56,25%	48
inc	1015	85	91,63%	48	56,47%	69
squar5	248	34	86,29%	22	64,71%	41
xor5	31	7	77,42%	2	28,57%	6
cordic	16777214	3898	99,98%	748	19,19%	89662

TABLE III.
EXPERIMENTAL RESULTS(3)

Benchmark Circuit	Factorial complexity of variable ordering problem					
	Before optimization	After optimization	Reduction ratio	Logic gates	Logic gate ratio	Time (ms)
con1	94	11	88,30%	5	45,46%	327
rd53	93	24	74,19%	10	41,67%	119
rd84	1020	70	93,14%	22	31,43%	64134
sqrt8	336	28	91,67%	12	42,86%	17065
9sym	511	31	93,93%	10	32,26%	162304
sao2	4092	95	97,68%	52	54,74%	8389400
5xp1	628	62	90,13%	18	29,03%	6090
misex1	473	44	90,70%	22	50,00%	4679
inc	1015	78	92,32%	44	56,41%	13622
squar5	248	33	86,69%	20	60,61%	496
xor5	31	7	77,42%	2	28,57%	63

TABLE IV.
EXPERIMENTAL RESULTS(4)

Benchmark Circuit	Number of nodes after optimization				
	Constant	Linear	Hillclimbing	Simulated Annealing	Factorial
con1	15	13	11	11	11
5xp1	89	76	82	64	62
Inc	90	85	81	78	78
misex1	50	48	48	44	44
sao2	188	154	146	100	95
sqrt8	39	37	37	28	28
squar5	38	34	38	33	33